

Optimization Algorithms & Linear Algebra for Deep Learning

Vatsav Reddy

Under the Guidance of
Prof. Praveen Parchuri

International Institute of Information Technology, Hyderabad

May 21, 2026

Contents

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Current Section

- 1 **Recap: Gradient Descent & Contour Maps**
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Gradient Descent Recap

Setup

Prediction $\hat{y} = f(x) = \frac{1}{1+e^{-(w \cdot x + b)}}$, parameter vector $w = \{w^1, w^2, \dots\}$, labels $q = \{q_1, \dots, q_h\}$, $\mathcal{L}(p \| q) = \text{CE}(p, q)$.

Update Rule: $w \leftarrow w - \eta \nabla w$

- **Steep:** $\frac{\Delta y_1}{\Delta x_1}$ large $\Rightarrow$ updates **large** $\Rightarrow$ **Fast**
- **Gentle:** $\frac{\Delta y_2}{\Delta x_2}$ small $\Rightarrow$ updates **small** $\Rightarrow$ **Slow**

Contour Maps (5.3)

Key Observation

The error surface can be visualised in 3-D (over w and b axes) or as 2-D contours. Moving 3D $\rightarrow$ 2D $\rightarrow$ “combination” view aids intuition.

Reading Contours

- Ellipse/circle boundary $\Rightarrow$ **same error**
- **Small** gap $\Rightarrow$ steep slope
- **Large** gap $\Rightarrow$ gentle slope
- Plateau $\Rightarrow$ very slow movement

Key Rule (proportional to gradient)

- Gentle $\rightarrow$ smaller gradient $\rightarrow$ **slower moment**
- Steep $\rightarrow$ higher gradient $\rightarrow$ **higher moment**

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 **Momentum-Based Descent**
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Problem with Vanilla Gradient Descent

Problem

Vanilla GD takes a lot of time to navigate regions with a **gentle slope** because the gradient is very small there.

Momentum GD — Intuition

*“If I am repeatedly being asked to move in the same direction, then I should probably gain some confidence and start taking **bigger steps** in that direction.”*

Regularisation Note

- **L1 (Lasso):** $\mathcal{L}_1 = \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j |w_j|$ (encourages *sparsity*)
- **L2 (Ridge):** $\mathcal{L}_2 = \sum_i (y_i - \hat{y}_i)^2 + \lambda \sum_j w_j^2$ (*non-sparse* / shrinkage)

Momentum Update Rule

Update Equations

$$\text{update}_t = \beta \cdot \text{update}_{t-1} + \eta \nabla w_t \quad \leftarrow \text{current gradient}$$

$$w_{t+1} = w_t - \text{update}_t = w_t - \beta \text{update}_{t-1} - \eta \nabla w_t$$

In addition to the current update we also look at the **history of updates**.

Exponentially Weighted Average — Derivation ($\text{update}_0 = 0$)

$$\text{update}_1 = \eta \nabla w_1$$

$$\text{update}_2 = \beta \eta \nabla w_1 + \eta \nabla w_2$$

$$\text{update}_3 = \beta^2 \eta \nabla w_1 + \beta \eta \nabla w_2 + \eta \nabla w_3$$

$$\vdots$$

$$\boxed{\text{update}_t = \beta^{t-1} \eta \nabla w_1 + \cdots + \eta \nabla w_t}$$

Why Momentum Works

Key Insight

Even in regions with **gentle slopes**, momentum-based GD can take **large steps** because the accumulated momentum carries it along.

Is Moving Fast Always Good?

No! Momentum can take many **U-turns** before converging. Despite these U-turns it still converges *faster* than vanilla GD.

Observation from Notes

The U-turns arise because momentum overshoots; the gradient corrects direction repeatedly. Convergence is still guaranteed and faster than plain GD.

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent**
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

NAG: Look Before You Leap

Momentum (standing here)

- 1 Stand here
- 2 Check slope *here*
- 3 Move

A single data point — current position only

Nesterov (standing ahead)

- 1 Peek ahead
- 2 Check **future** slope
- 3 Adjust *before* moving

NAG — Key Difference & Update Rule

Compute the gradient at the **look-ahead** value of w :

$$w_{\text{look-ahead}} = w_t - \beta \text{update}_{t-1}$$

(Move according to history first — you might end up before a cliff!)

$$\text{update}_t = \beta \text{update}_{t-1} + \eta \nabla w_{\text{look-ahead}}$$

$$w_{t+1} = w_t - \text{update}_t$$

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent**
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Stochastic Gradient Descent (SGD)

Consequence of Full-batch GD

Full-batch requires **millions of calculations** per update step (one pass over all data before any parameter update).

Algorithm 1 Stochastic Gradient Descent (update per point)

Input: $w, b, \eta, \text{max_epoch}$

```
1: for  $i$  in range(max_epoch) do  
2:    $dw, db \leftarrow 0, 0$   
3:   for  $x, y$  in zip( $X, Y$ ) do  
4:      $dw = \text{grad}_w(w, b, x, y); \quad db = \text{grad}_b(w, b, x, y)$   
5:      $w \leftarrow w - \eta dw; \quad b \leftarrow b - \eta db$   
6:   end for  
7: end for
```

Stochastic = Like Tossing a Coin

- **No theoretical guarantee** that each step decreases the loss
- Many **oscillations** — greedy, per-point decisions
- Each point pushes parameters in the direction most favourable to *it*, ignoring all other points
- Data points effectively **compete** with each other

Mini-batch Gradient Descent (MBGD)

Idea: Better Gradient Estimate

SGD estimates the gradient from **1 data point** at a time. MBGD averages over a **batch of B points**.

Algorithm 2 Mini-batch Gradient Descent

Input: w, b, η ; min_batch_size B

```
1: for  $i$  in range(max_epoch) do
2:    $dw, db, \text{num\_seen} \leftarrow 0, 0, 0$ 
3:   for  $x, y$  in zip( $X, Y$ ) do
4:      $dw \leftarrow dw + \text{grad}_w(w, b, x, y)$ ;  $db \leftarrow db + \text{grad}_b(w, b, x, y)$ 
5:      $\text{num\_seen} \leftarrow \text{num\_seen} + 1$ 
6:     if  $\text{num\_seen} \bmod B = 0$  then
7:        $w \leftarrow w - \eta dw$ ;  $b \leftarrow b - \eta db$ ;  $dw, db \leftarrow 0, 0$ 
8:     end if
9:   end for
10: end for
```

Why MBGD Oscillates Less

Coin-Toss Analogy

Tossing a coin k times gives a better estimate of $P(\text{heads})$ than a single toss.

1 toss : $\frac{1}{2} \mid \frac{1}{2}$ k tosses : know which is the better estimate

Higher $k \Rightarrow$ better estimate $\Rightarrow$ fewer oscillations.

Similarly: larger mini-batch $\Rightarrow$ better gradient estimate $\Rightarrow$ **MBGD oscillates less than SGD.**

Relative Oscillations (large $\rightarrow$ small)

Stochastic GD $>$ MBGD $>$ Vanilla GD

MBGD takes relatively *shorter* U-turns than pure SGD.

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum**
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Choosing the Learning Rate

The Dilemma

Set η high to “blow up” the small gradient on gentle slopes $\Rightarrow$ steep regions diverge!

Adaptive η Philosophy

Gentle slope $\Rightarrow$ fast (large η)

Steep slope $\Rightarrow$ slow (small η)

- Typical values: $\eta \in \{0.0001, 0.001, 0.01, \mathbf{0.1}\}$
- Fine search on **log scale**: $0.05 \rightarrow 0.1 \rightarrow 0.2 \rightarrow \dots$
- These are **heuristics** — no single winning strategy
- High η : faster convergence, risk of overshooting
Low η : slower but more precise

Multi-start Strategy

Every time we reach a minimum, save (w_0, b_0) . Take the minimum which is *less lossy*.

Annealing the Learning Rate

Step Decay

- Halve η after every T epochs
- Halve η after an epoch if validation error $>$ previous epoch's error
- Save model at epoch t ; if epoch $t+1$ loss $>$ epoch t loss, restore saved model and set $\eta \leftarrow \eta/2$

Exponential Decay

$$\eta = \eta_0 e^{-kt} \quad (\eta_0, k \text{ are hyperparameters; } t \text{ is step number})$$

$1/t$ Decay

$$\eta = \frac{\eta_0}{1 + kt}$$

Better not to use — introduces an extra hyperparameter ($1\frac{1}{2}$ more than needed).

Tips for Momentum

Schedule (Sutskever et al. 2013)

$$\mu_{t+1} = \min\left(1 - 2^{-1 - \log_2(\lfloor t/250 \rfloor + 1)}, \mu_{\max}\right)$$

- $\mu_0 = 0.5, \quad \mu_{250} = 0.75, \quad \mu_{750} = 0.875$
- $\mu_{\max}$ chosen from $\{0.999, 0.995, 0.99, 0.9, 0\}$

The number of effective updates $\approx \mu_t + \frac{d\ell}{dw}$.

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search**
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Line Search

Goal

Find the best η *at each step* from a discrete set:

$$\eta \in \{0.1, 0.5, 1.0, 5.0, 10.0\}$$

- Gentle slope $\Rightarrow$ pick larger η
- Steep slope $\Rightarrow$ pick small η

Algorithm 3 Line Search

```
1: best  $w, b \leftarrow$  worst error
2: for  $\eta$  in  $\eta_{\text{tas}}$  do
3:    $\text{tmp}w = w - \eta \, dw$ ;  $\text{tmp}b = b - \eta \, db$ 
4:   if  $\text{error}(\text{tmp}w, \text{tmp}b) < \text{min error}$  then
5:     best  $w \leftarrow \text{tmp}w$ ; best  $b \leftarrow \text{tmp}b$ 
6:   end if
7: end for
```

Flip Side

If k η 's are tried: $k \times$ more computation. However, it avoids getting stuck in a local minimum for a particular learning rate — with another rate we might have reached a better region.

When Do Oscillations Happen?

- Oscillations occur when the loss *increases* from its current value
- Whether this happens depends on the learning rate chosen
- Convergence is **faster** than vanilla GD even with oscillations
- The oscillations in line search differ from those in Momentum and NAG — here they arise from the discrete η set, not from gradient history

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate**
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Motivation: Sparse Features

Problem Setup

$$\nabla w^1 = (f(x) - y) \cdot f(x) \cdot (1 - f(x)) \cdot x^1$$

If feature x^1 is **sparse** (mostly 0): $\nabla w^1 \approx 0$ for most inputs $\Rightarrow w^1$ gets very few updates.

Issue

- Sparse $\Rightarrow$ update slower; Dense $\Rightarrow$ update faster
- If x^1 is both sparse *and* important, we'd want to take updates to w^1 more seriously
- Very common in large NNs with thousands of features (e.g. credit fraud, education data)

Intuition for the Fix

Decay the learning rate in proportion to update history:

$$\eta \propto \frac{1}{\text{update history}}$$

Frequent feature $\Rightarrow$ give $\eta \downarrow$; Infrequent feature $\Rightarrow$ give $\eta \uparrow$.

AdaGrad Update Rule

$$v_t = v_{t-1} + (\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

- Frequent feature $\Rightarrow v_t$ large $\Rightarrow$ effective $\eta \downarrow$
- Infrequent feature $\Rightarrow v_t$ small $\Rightarrow$ effective η stays large $\uparrow$

Behaviour (100 random (x, y) pairs, 80% sparse)

Initially all algorithms (GD, Momentum, AdaGrad) move mainly along the **vertical** (b) **axis**; very little movement along the horizontal (w) axis.

Why? Because the feature for w is sparse, so w undergoes few updates; b is dense and updates frequently.

Flip Side of AdaGrad

$\sqrt{v_t}$ grows unboundedly (accumulates *all* past squared gradients). The effective η for b **decays to zero** — AdaGrad gets *stuck* near convergence.

Idea: Exponential Moving Average (EMA) of Squared Gradients

Instead of accumulating *all* past squared gradients, decay them with $\beta > 0.9$:

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla w_t)^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \nabla w_t$$

- $v_1 = (1 - \beta)(\nabla w_1)^2$
- Still accumulates history, but *not* aggressively (only a fraction each step)
- **Does not kill** the learning rate

Result

RMSProp overcomes AdaGrad's decay problem by being less aggressive. AdaGrad got stuck near convergence (unable to move in b direction). RMSProp **does not kill the learning rate**.

Adam: Adaptive Moment Estimation

Adam = RMSProp + Momentum + Bias Correction

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla w_t \quad \leftarrow \text{1st moment (mean / direction)}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla w_t)^2 \quad \leftarrow \text{2nd moment (adaptive scaling)}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \leftarrow \text{bias correction}$$

$$\beta_1 = 0.9, \quad \beta_2 = 0.99$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \cdot \hat{m}_t$$

Why Bias Correction in Adam?

Derivation via EMA

Treat stochastic gradients $g_t = \nabla w_t$ as random variables. We want the update to be close to $\mathbb{E}[g_t]$.

$$m_t = (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i$$

$$\mathbb{E}[m_t] = \mathbb{E}[g_t] \cdot (1 - \beta) \sum_{i=1}^t \beta^{t-i} = \mathbb{E}[g_t] \cdot (1 - \beta^t)$$

(Assumes all g_i from the same distribution, i.e. $\mathbb{E}[g_i] = \mathbb{E}[g_t]$.)

We **want** $\mathbb{E}[m_t] = \mathbb{E}[g_t]$, so divide by $(1 - \beta^t)$:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \Rightarrow \mathbb{E}[\hat{m}_t] = \mathbb{E}[g_t]$$

The same argument applies to $\hat{v}_t = v_t / (1 - \beta_2^t)$.

AdamW (Transformer Variant)

AdamW: Weight Decay via the Gradient System

In standard Adam: $g_t = \nabla_w \mathcal{L}(w_t)$.

In **AdamW** (used in Transformers), weight decay is added directly to the gradient:

$$g_t \leftarrow \nabla_w \mathcal{L}(w_t) + \lambda w_t$$

This yields:

$$w_{t+1} = w_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \eta \lambda w_t$$

The extra $-\eta \lambda w_t$ term gives true **weight-decay** behaviour: it optimises the loss and independently shrinks the weights.

Adam vs AdamW

In Adam, λw_t goes inside the moment estimation, changing the adaptive scaling. In AdamW, λw_t is applied *after* the adaptive step — cleaner decoupling.

Optimizer Comparison

Hierarchy of Preference (in practice)

Optimizer	Notes
GD	Baseline
GD + Momentum	Better for gentle slopes
NAG	“Look before you leap”
AdaGrad	Adaptive η ; kills learning rate over time
RMSProp	Fixes AdaGrad decay via EMA
Adam	Default choice ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$)

Caveats

- Adam does *not* actually converge in some real-world cases — some recent work confirms this
- For sequence generation: $\eta = 0.001$ or 0.0001 work best
- SGD + Momentum (Nesterov/Classical) with simple annealing also works well in practice
- Adam *might* be the best choice, but is not guaranteed to be!

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors**
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Basis & Linear Independence

Basis

A set of vectors $\in \mathbb{R}^n$ is a **basis** if they are linearly independent and every vector $\in \mathbb{R}^n$ can be expressed as a linear combination of them.

Linearly Independent Vectors

$v_1, \dots, v_n$ are linearly independent iff the only solution to

$$c_1 v_1 + c_2 v_2 + \dots + c_n v_n = \mathbf{0}$$

is $c_1 = c_2 = \dots = c_n = 0$.

Example in $\mathbb{R}^2$

$x = [1, 0]^T$ and $y = [0, 1]^T$ are the standard unit vectors. Any $[a, b]^T \in \mathbb{R}^2$ equals $a[1, 0]^T + b[0, 1]^T$.

Nothing is sacrosanct about this choice — we could equally use $[2, 3]^T$ and $[5, 7]^T$ as basis vectors. In general, any n linearly independent vectors in $\mathbb{R}^n$ form a basis.

Orthonormal Basis

Representing Any Vector

$$z = \alpha_1 u_1 + \alpha_2 u_2 + \cdots + \alpha_n u_n$$

Solving for the α_i via Gaussian elimination: $O(n^3)$.

Orthonormal = Orthogonal + Normal (unit vectors)

$$u_i^T u_j = 0 \quad \forall i \neq j, \quad u_i^T u_i = \|u_i\|^2 = 1$$

With an orthonormal basis we can **directly** find each α_i :

$$\alpha_i = z^T u_i$$

Linear complexity $O(N)$ per coefficient $\Rightarrow$ total $O(Nn)$ vs. $O(n^3)$.

Orthonormal basis is the most convenient basis!

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors**
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Eigenvalues & Eigenvectors

Motivation

When a matrix A multiplies a general vector, the vector **strays from its direction** — it may get rotated, scaled, or shortened. For a given square A there exist **special vectors that refuse to stray**: they only get scaled.

Definition

$$Ax = \lambda x$$

The vector changes magnitude only, **not direction**. x is an **eigenvector**; λ is its **eigenvalue**.

Theorem 1

Eigenvectors of $A \in \mathbb{R}^{n \times n}$ with **distinct eigenvalues** are **linearly independent** $\Rightarrow$ form a basis.

Theorem 2

Eigenvectors of a **square symmetric** matrix are **orthogonal**.

Applications

PCA, Autoencoders, SVD word vectors, PageRank (limelight), spectral graph theory.

Why Use Eigenvectors as a Basis?

Key Motivation for PCA

A square symmetric matrix gives a very convenient basis: its eigenvectors are **orthogonal** (Thm 2) and form a **linearly independent set** (Thm 1).

Optimisation Connection

For square symmetric $n \times n$ matrix A :

$$\max_x x^T A x \quad \text{s.t. } \|x\| = 1 \quad \Rightarrow \text{eigenvector of } \mathbf{largest} \text{ eigenvalue}$$

$$\min_x x^T A x \quad \text{s.t. } \|x\| = 1 \quad \Rightarrow \text{eigenvector of } \mathbf{smallest} \text{ eigenvalue}$$

Proof: Lagrangian $L = x^T A x - \lambda(x^T x - 1)$; $\frac{dL}{dx} = 2Ax - 2\lambda x = 0 \Rightarrow Ax = \lambda x$.

Dominant Eigenvalue & Steady State

Stochastic Matrix

A matrix M is **stochastic** if all entries are positive and every *column* sums to 1.

Theorem 1 (Dominant Eigenvalue of a Stochastic Matrix)

The largest (dominant) eigenvalue of a stochastic matrix is 1.

Theorem 2

*If A is $n \times n$ with dominant eigenvalue d_1 , then $Ax_0, A^2x_0, A^3x_0, \dots$ approaches a **multiple of the dominant eigenvector** of A . In general: $A^p x_0 = k |d_1|^p e_{d_1}$.*

Sequence Behaviour

- $|\lambda| > 1 \Rightarrow$ sequence **explodes**
- $|\lambda| < 1 \Rightarrow$ sequence **vanishes**
- $|\lambda| = 1 \Rightarrow$ sequence reaches a **steady state**

Stochastic Matrix Example: Two-Restaurant Problem

Setup

$$V_{(0)} = \begin{bmatrix} k_1 \\ k_2 \end{bmatrix}, \quad M = \begin{bmatrix} p & 1-q \\ 1-p & q \end{bmatrix}$$

Fraction p of Chinese-restaurant students stay the next day; $(1-p)$ switch. Fraction q of Mexican-restaurant students stay; $(1-q)$ switch.

$V_{(n)} = M^n V_{(0)}$ — will the population reach a **steady state**?

Answer via Eigenvalues

M is stochastic $\Rightarrow$ dominant eigenvalue $= 1 \Rightarrow V_{(n)} \rightarrow k \cdot e_d$ (a multiple of the dominant eigenvector). **Population becomes constant after some step n .**

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)**
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

Eigenvalue Decomposition

Construction

Let $u_1, \dots, u_n$ be eigenvectors of A and $d_1, \dots, d_n$ the corresponding eigenvalues. Let U have columns $u_1, \dots, u_n$:

$$AU = [Au_1 \ \cdots \ Au_n] = [d_1 u_1 \ \cdots \ d_n u_n] = U\Lambda$$

$$\boxed{A = U\Lambda U^{-1}} \quad (\text{Eigenvalue Decomposition})$$

$$\Lambda = U^{-1}AU \quad (\text{Diagonalisation of } A)$$

Symmetric Case

If A is symmetric: eigenvectors are orthogonal and normalised, so $U^T U = I \Rightarrow U^{-1} = U^T$:

$$A = U\Lambda U^T$$

Sequence $x_0, Ax_0, A^2x_0, \dots$ in terms of EVD: $U\Lambda^n U^T x_0$ — explodes if $|\lambda| > 1$, steady if $|\lambda| = 1$, vanishes if $|\lambda| < 1$.

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations**
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

PCA: Motivation

Goal

Represent data using **fewer dimensions** such that:

- Data has **high variance** along these dimensions
- Dimensions are **linearly independent** (uncorrelated)
- Even better: dimensions are **orthogonal** (most convenient basis)

Covariance

$$c_{ij} = \frac{1}{m} \sum_{l=1}^m (x_{li} - \mu_i)(x_{lj} - \mu_j) = \frac{1}{m} (X^T X)_{ij} \quad (\mu_i = 0 \text{ for zero-mean data})$$

- +cov: both increase/decrease together
- -cov: one increases as the other decreases
- ≈ 0 : no linear relationship

Correlation

Normalised covariance; direction -1 to $+1$ and strength. $+1$: perfect positive; 0 : no linear relationship; -1 : perfect negative. High correlation $\Rightarrow$ one variable is redundant.

PCA: Mathematical Formulation

Setup

Data matrix $X \in \mathbb{R}^{m \times n}$ (zero-mean, unit variance). Let $P = [p_1 \cdots p_n]$ be an $n \times n$ orthonormal matrix.

$$\hat{X} = XP \quad (\text{projection onto new basis})$$

For an orthonormal basis: $\alpha_{ij} = x_i^T p_j$.

Covariance of Transformed Data

Theorem: If X has zero-mean columns then so does $\hat{X} = XP$.

Theorem: $X^T X$ is symmetric.

$$\frac{1}{m} \hat{X}^T \hat{X} = \frac{1}{m} P^T X^T X P = P^T \Sigma P, \quad \Sigma = \frac{1}{m} X^T X$$

We want $P^T \Sigma P = D$ (diagonal):

$$\left(\frac{1}{m} \hat{X}^T \hat{X} \right)_{ij} = 0 \ (i \neq j) \text{ (zero covariance), } \neq 0 \ (i = j) \text{ (non-zero variance)}$$

PCA: The Answer

Which P Diagonalises Σ ?

The matrix P whose columns are the **eigenvectors** of $\Sigma = \frac{1}{m}X^T X$:

$$\Sigma = P\Lambda P^T \Rightarrow P^T \Sigma P = \Lambda \text{ (diagonal)}$$

The new basis P is the **eigenvector basis** of $X^T X$.

Why Is This a Good Basis?

- Eigenvectors of $X^T X$ are **linearly independent** (Thm 1)
- Eigenvectors of $X^T X$ are **orthogonal** (since $X^T X$ is symmetric; Thm 2)
- This is **Principal Component Analysis (PCA)**: transforms data to a basis with low covariance and high variance dimensions
- In practice, keep only the **top- k** dimensions (high eigenvalue $\Rightarrow$ high variance)

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)**
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary

PCA Interpretation 2: Minimising Reconstruction Error

Setup

Given n orthonormal vectors $p_1, \dots, p_n$ we can represent each x_i exactly:

$$x_i = \sum_{j=1}^n \alpha_{ij} p_j$$

Keep only the **top- k dimensions**:

$$\hat{x}_i = \sum_{j=1}^k \alpha_{ij} p_j$$

Select p_j 's to minimise reconstruction error:

$$e = \sum_{i=1}^m (x_i - \hat{x}_i)^T (x_i - \hat{x}_i)$$

Result (using orthonormality)

After simplification:

$$e = \sum_{j=k+1}^n p_j^T \Sigma p_j$$

Minimising $e \Rightarrow$ select $p_{k+1}, \dots, p_n$ as the **smallest eigenvectors** of Σ , i.e. **keep the top- k eigenvectors**. The discarded $n - k$ directions are those with minimum variance.

PCA Interpretation 2: Worked Example

Example in $\mathbb{R}^2$ (data along $y = x$)

New basis: $u_1 = \left[\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right]^T$ and $u_2 = \left[-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right]^T$.

Consider $x = [3.3, 3]$:

$$\alpha_1 = x^T u_1 = \frac{6.3}{\sqrt{2}}, \quad \alpha_2 = x^T u_2 = \frac{0.3}{\sqrt{2}}$$

Perfect reconstruction (2-D): $\hat{x} = \alpha_1 u_1 + \alpha_2 u_2 = [3.3, 3]$.

Using top-1 only (ignoring low-variance u_2):

$$\hat{x} = \alpha_1 u_1 = [3.15, 3.15] \quad (\text{minimum reconstruction error})$$

Small $\alpha_2 \Rightarrow$ discarding u_2 introduces little error.

Key Idea

PCA uses only the top- k eigenvectors:

- 1 Covariance between new dimensions is 0 (diagonalise covariance matrix)
- 2 Discarded dimensions are those with minimum variance (minimum reconstruction error)

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)**
- 14 PCA: Complete Summary
- 15 Summary

PCA Interpretation 3: Maximising Variance

Variance along the i -th Dimension

The i -th dimension of the transformed data is $\hat{X}_i = Xp_i$. Its variance:

$$\frac{\hat{X}_i^T \hat{X}_i}{m} = \frac{1}{m} p_i^T X^T X p_i = \frac{1}{m} p_i^T \lambda_i p_i = \frac{\lambda_i}{m}$$

(since p_i is an eigenvector of $X^T X$ with eigenvalue λ_i , and $p_i^T p_i = 1$).

Conclusion

Variance along dimension i is **scaled by eigenvalue** λ_i .

Discarding low-eigenvalue dimensions $\Rightarrow$ discarding **lowest-variance** (most noisy, least informative) dimensions. $\Rightarrow$ We did the right thing!

Quick Summary: Three Interpretations of PCA

- 1 **Covariance**: covariance between new dimensions is **minimised** (diagonalise Σ)
- 2 **Variance**: dimensions are chosen to have **high variance**
- 3 **Compression**: data representable with **fewer dimensions** (minimum reconstruction error)

PCA in Practice: Eigenfaces

Setup

$X \in \mathbb{R}^{m \times 10K}$ (100×100 images = 10K dimensions).

Retain **top 50–100 eigenvectors** of $X^T X$:

- $X^T X$ is $n \times n$ ($n = 10K$); each eigenvector is n -dimensional
- Convert each eigenvector to a 100×100 image $\Rightarrow$ **eigenface**
- These form a basis for any face in the dataset

Benefits

- Any face = linear combination of eigenfaces
- Store $p_1, \dots, p_k$ once; then only scalars $\alpha_{i1}, \dots, \alpha_{ik}$ per image
- **Data compression**: significant storage savings with little quality loss
- Multiple images of the same person (different lighting, contrast) are *closer* in the new space

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary**
- 15 Summary

PCA: Complete Summary — Setup & Goal

The Problem We Solve

Given m data points in $\mathbb{R}^n$ (zero-mean), find a new **orthonormal basis** of size $k \ll n$ such that:

- 1 Dimensions are **uncorrelated** (zero off-diagonal covariance)
- 2 Data exhibits **maximum variance** in these directions
- 3 **Reconstruction error** from the k -dim approximation is minimised

One Algorithm, Three Views

Interpretation	Optimises	By
Decorrelation	min off-diagonal covariance	diagonalise Σ
Max Variance	max variance per dimension	keep top eigenvalues
Min Error	min reconstruction error	discard small eigenvectors

All three views give the same answer: use the eigenvectors of $\Sigma = \frac{1}{m}X^T X$.

PCA: Complete Summary — The Algorithm

Step-by-Step

- 1 **Centre** the data: $x_i \leftarrow x_i - \mu$
- 2 **Compute** covariance matrix $\Sigma = \frac{1}{m} X^T X \in \mathbb{R}^{n \times n}$
- 3 **Eigendecompose**: $\Sigma = P \Lambda P^T$ (columns of P are eigenvectors; diagonal of Λ are eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$)
- 4 **Select** top- k eigenvectors $P_k = [p_1, \dots, p_k] \in \mathbb{R}^{n \times k}$
- 5 **Project**: $\hat{X} = X P_k \in \mathbb{R}^{m \times k}$

Key Properties of the Transformed Data

- $\hat{X}$ has **zero mean** (inherited from X)
- Covariance of $\hat{X}$: $\frac{1}{m} \hat{X}^T \hat{X} = \Lambda_k$ (diagonal $\Rightarrow$ uncorrelated dimensions)
- Variance along i -th new dimension $= \lambda_i / m$
- Fraction of variance retained $= \sum_{i=1}^k \lambda_i / \sum_{i=1}^n \lambda_i$

PCA: Complete Summary — Why Eigenvectors?

Theorem 1

Eigenvectors of A with **distinct eigenvalues** are **linearly independent**.

$\Rightarrow$ They form a basis.

Theorem 2

Eigenvectors of a **symmetric** matrix are **orthogonal**.

$\Rightarrow \Sigma = \frac{1}{m}X^T X$ is symmetric, so its eigenvectors are orthogonal.

Optimisation View

$$\max_{p: \|p\|=1} p^T \Sigma p = \lambda_{\max} \quad (\text{achieved by top eigenvector})$$

Each subsequent direction maximises remaining variance subject to being orthogonal to all previous directions.

Discard Low-Eigenvalue Directions Because...

- Low eigenvalue $\Rightarrow$ low variance $\Rightarrow$ mostly noise
- Low eigenvalue $\Rightarrow$ small $\alpha_{ij} \Rightarrow$ small reconstruction error if dropped

PCA: Complete Summary — Connections & Applications

Connections to Other Concepts

- **EVD:** $\Sigma = P\Lambda P^T$ is exactly EVD of a symmetric matrix; $P^{-1} = P^T$
- **Orthonormal basis:** eigenvectors of a symmetric matrix are orthogonal $\Rightarrow$ finding coordinates is $O(N)$ per dimension
- **SVD:** PCA of X is equivalent to SVD of X ; eigenvectors of $X^T X$ are the right singular vectors
- **Stochastic matrix:** dominant eigenvector gives the steady state distribution (same maths, different context)

Applications

- **Eigenfaces:** face recognition using top-100 eigenvectors of $X^T X$; any face is a linear combination of eigenfaces
- **Data compression:** store k scalars per image instead of $n = 10\text{K}$ pixels
- **Pre-processing for classifiers:** uncorrelated, high-variance features improve generalisation
- **Noise reduction:** discarding low-variance dimensions removes uninformative noise

Current Section

- 1 Recap: Gradient Descent & Contour Maps
- 2 Momentum-Based Descent
- 3 Nesterov Accelerated Gradient Descent
- 4 Stochastic & Mini-batch Gradient Descent
- 5 Tips for Learning Rate & Momentum
- 6 Line Search
- 7 Gradient Descent with Adaptive Learning Rate
- 8 Linear Algebra — Basis & Orthonormal Vectors
- 9 Eigenvalues & Eigenvectors
- 10 Eigenvalue Decomposition (EVD)
- 11 PCA & Its Interpretations
- 12 PCA — Interpretation 2 (Reconstruction Error)
- 13 PCA — Interpretation 3 (Variance)
- 14 PCA: Complete Summary
- 15 Summary**

Summary

Optimization Algorithms

- **Momentum**: exponentially weighted gradient history; fast on gentle slopes; U-turns but faster than vanilla GD
- **NAG**: “look before you leap”; checks future slope
- **SGD / MBGD**: stochastic; fewer calculations; mini-batch reduces oscillations
- **AdaGrad**: adaptive η ; fixes sparse features; kills η via unbounded accumulation
- **RMSProp**: EMA decay; fixes AdaGrad
- **Adam**: RMSProp + Momentum + bias correction; **default choice**
- **AdamW**: Adam + weight decay in gradient (Transformers)

Linear Algebra for DL

- **Basis**: linearly independent spanning set
- **Orthonormal basis**: most convenient; $O(N)$ projection
- **Eigenvectors**: $Ax = \lambda x$; symmetric $\Rightarrow$ orthogonal
- **EVD**: $A = U\Lambda U^{-1}$; symmetric:
 $A = U\Lambda U^T$
- **Stochastic matrix**: dominant eigenvalue is 1; steady state exists
- **PCA**: 3 interpretations: decorrelation, max variance, min reconstruction error
- **Eigenfaces**: PCA for face compression

“Gentle slope $\rightarrow$ slow GD, fast momentum; Steep slope $\rightarrow$ fast GD, large gradient, careful η .”

Thank You

Questions?